

Cluster and Cloud Computing – Lectures Week 2
Distributed and Parallel Computing Systems

Professor Richard O. Sinnott
Director, Melbourne eResearch Group
University of Melbourne
rsinnott@unimelb.edu.au

Notice



Wanting to connect with industry leaders and other students?

Feeling lost about what careers are out there or where you want to go next?

Curious to learn about the mining & resources industry?



Come along to network, explore career paths and job roles, or simply listen and learn.



RSVP HERE!



WHEN:
FROM 6:30PM,
18 MARCH 2026

WHERE:
RYDGES MELBOURNE



This annual event gives all students, no matter their course or stage of study, the chance to connect with industry leaders and early-career professionals in a relaxed, social setting.



Come along to our Students Meet Industry Night, to get the chance to speak with our partners!

6:30PM 18 MARCH 2026 | RYDGES MELBOURNE

RSVP HERE!



Some things...

- Next weeks lectures/workshops
 - (it starts getting practical!!!)
- Assignment 1
 - Attending next weeks lecture/workshops essential to do assignment
 - Find a “buddy”
 - See google sheet
 - https://docs.google.com/spreadsheets/d/1Mwsbpbend1EJ_5DVS9RHfvU2Zy_1O0xvUh9gACpLBWg/edit?gid=0#gid=0

Some more things...

- Assignment 2 – teams reminder
 - Organise or be organised
 - Find many buddies
 - See googlesheet
 - <https://docs.google.com/spreadsheets/d/182GLdOmr2-4Rs3LDMu2FbIXDdMKx10ivUZWfuSKvIgA/edit?gid=0#gid=0>
- UniMelb data centre visit
 - See googlesheet
 - https://docs.google.com/spreadsheets/d/13Jjq5sOPWf1JkZyWxnZE_Qx9b7qmtmhAUY_KQ1CoPhM/edit?gid=0#gid=0

SPARTAN

- The process:

- You should go to Karaage (<https://dashboard.hpc.unimelb.edu.au/karaage>)
- Click Apply for account, and enter your Unimelb student email address
- You will get a link sent in an email to continue with your account creation
- When prompted to either create a new project, or join an existing one, select join an existing project, and apply to join the project COMP90024
- Note that you pick the password you want – this is the one you use for SSH access (see below)
 - This may or may not be the same as your UniMelb password

- Access and use will look like

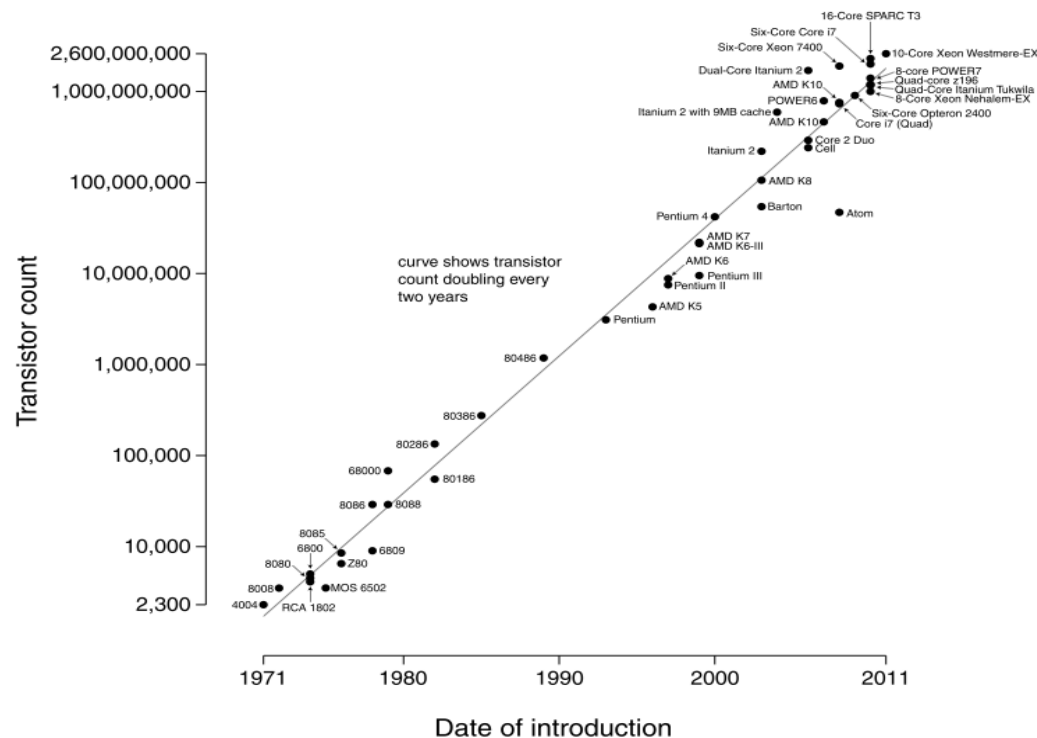
- ssh rsinnott@spartan.hpc.unimelb.edu.au
- *Enter the password you selected above!*
 - (may need Putty or VPN if coming in from outside University)
- Note that the accounts are managed by HPC teams and not me!

Do NOT do "*stupid stuff*" on SPARTAN!!!
All access and use is strictly monitored!!!

Compute Scaling

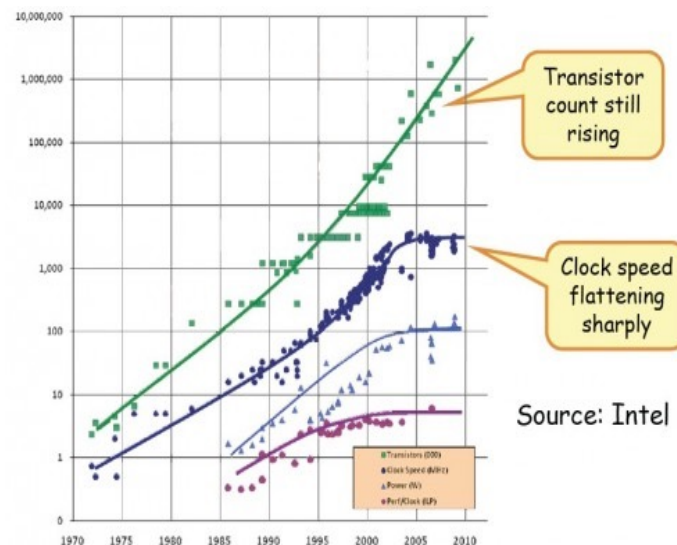
- Vertical Computational Scaling
 - Have faster processors
 - Switch your n GHz CPU for a $2n$ GHz one = 2x faster
 - Easy to do, but costs more
 - Limits of fundamental physics/matter (nanoCMOS)

Microprocessor Transistor Counts 1971-2011 & Moore's Law



Compute Scaling

- Horizontal Computational Scaling
 - Have more processors
 - Easy to **add more**; cost increase not so great
 - But...
 - harder to design, develop, test, debug, deploy, manage, understand, ...



- Note that for many/most folks
 - HTC far more important than HPC
 - (I don't do HPC, but I've done a lot of HTC)

Add More = ???

- Single machine multiple cores
 - Typical laptop/PC/server these days
- Loosely coupled collection/cluster of machines
 - Pooling/sharing of resources
 - Dedicated vs available only when not in use by others
 - Web services, ... Condor, Seti@home, Boinc
- Tightly coupled cluster of machines
 - Typical HPC/HTC set-up (SPARTAN, NCI, ...)
 - Many servers in same rack/server room (often with fast message passing interconnects)
- Widely distributed clusters of machines
 - UK NGS, EGEE, ... distributed systems more generally
- Hybrid combinations of the above
 - Leads to many challenges with distributed systems
 - Shared state (or lack thereof)
 - Message passing paradigms – dangers of delayed/lost messages

Add More...limitations

- Add more processes...?
 - If n processors (cores) are thrown at a problem how much faster will it go?
- Some terminology:
 - $T(1)$ = time for serial computation
 - $T(N)$ = time for N parallel computations
 - $S(N)$ = speed up

$$S(N) = \frac{T(1)}{T(N)}$$

- Proportion of speed up depends on parts of program that can't be parallelised



Amdahl's Law

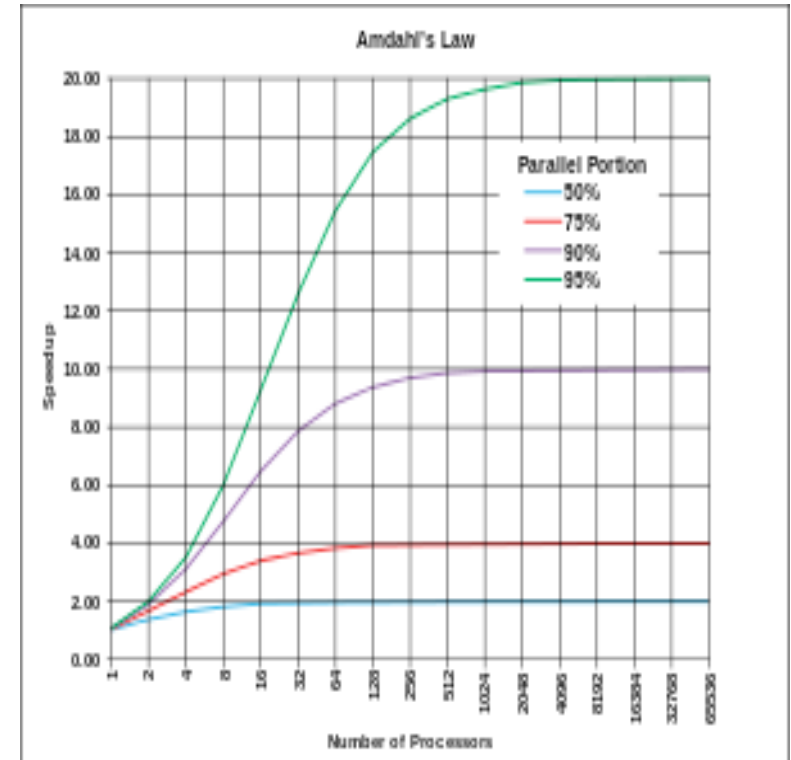
α = Fraction of program that can be done in parallel

$1 - \alpha$ = fraction that must be carried out on single CPU

T = time needed for application to execute on a single CPU

SpeedUp based on $T_{\text{seq}}/T_{\text{par}} = \frac{T}{(1 - \alpha)T + \alpha(T/N)}$

When N gets very large, $T_{\text{seq}}/T_{\text{par}}$ tends to $\frac{1}{1 - \alpha}$



Thus if 95% of the program can be parallelized ($1/(1 - 0.95) = 20$), the theoretical maximum speedup using parallel computing would be $20\times$, no matter how many processors are used, i.e., if the non-parallelisable part takes 1 hour, then no matter how many cores you throw at it, it won't complete in <1 hour.

Over-Simplification of Amdahl's Law

- Consider a program that executes a single loop, where all iterations can be computed independently, i.e., code can be parallelized. By splitting the loop into several parts, e.g., one loop iteration per processor, each processor now has to deal with loop overheads such as calculation of bounds, test for loop completion etc. This overhead is replicated as many times as there are processors. In effect, loop overhead acts as a further (serial) overhead in running the code. Also getting data to/from many processor overheads?

In a nutshell Amdahl's Law greatly simplifies the real world!

- It also assumes a fixed problem size – sometimes can't predict length of time required for jobs,
 - e.g., state space exploration or differential equations that don't solve...



Gustafson-Barsis's Law

- Gives the “scaled speed-up”

Let T be the total time a parallel program requires when using N processors

Let α be the fraction of execution time the program spends executing code in parallel

Scaled SpeedUp $S =$

$$\frac{(1 - \alpha)T + \alpha \cdot T \cdot N}{T} = (1 - \alpha) + \alpha N$$

Speed up S using N processes is given as a linear formula dependent on the number of processes and the fraction of time to run sequential parts. Gustafson's Law proposes that programmers tend to set the size of problems to use the available equipment to solve problems within a practical fixed time. Faster (more parallel) equipment available, larger problems can be solved in the same time.

Metaphors

The [Wikipedia page for Gustafson's Law](#) offers the following metaphor to contrast the two laws.

Amdahl's Law approximately suggests: Suppose a car is traveling between two cities 60 miles apart, and has already spent one hour traveling half the distance at 30 mph. No matter how fast you drive the last half, it is impossible to achieve 90 mph average before reaching the second city. Since it has already taken you 1 hour and you only have a distance of 60 miles total; going infinitely fast you would only achieve 60 mph.

Gustafson-Barsis's Law approximately suggests: Suppose a car has already been traveling for some time at less than 90mph. Given enough time and distance to travel, the car's average speed can always eventually reach 90mph, no matter how long or how slowly it has already traveled. For example, if the car spent one hour at 30 mph, it could achieve this by driving at 120 mph for two additional hours, or at 150 mph for an hour.

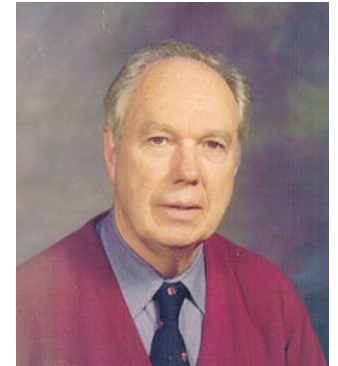
Assignment 1

- Does the 1:8 and/or 2:4 scenario go 8 times faster than the 1:1 case?
- Static data and predictable/known compute demands
 - Wall time (more next week!)
 - What if not a static file(s) but continually growing file(s)?
- *In principle*, solutions could be used to process an infinite amount of data

Computer Architecture - 101

- At the simplest level a computer comprises
 - CPU for executing programs
 - ALU, FPU, ...
 - Load/store unit
 - Registers (fast memory locations)
 - Program counter (address of instruction that is executing)
 - Memory interface
 - Memory that stores/executing programs and related data
 - I/O systems (keyboards, networks, ...)
 - Permanent storage for read/writing data into out of memory
 - Of key importance (especially for HPC systems!) is the balance of all of these
 - Superfast CPUs starved of data
 - There are many different ways to design/architect computers
 - different flavours suitable to different problems

Computer Architectures



- Flynn's Taxonomy

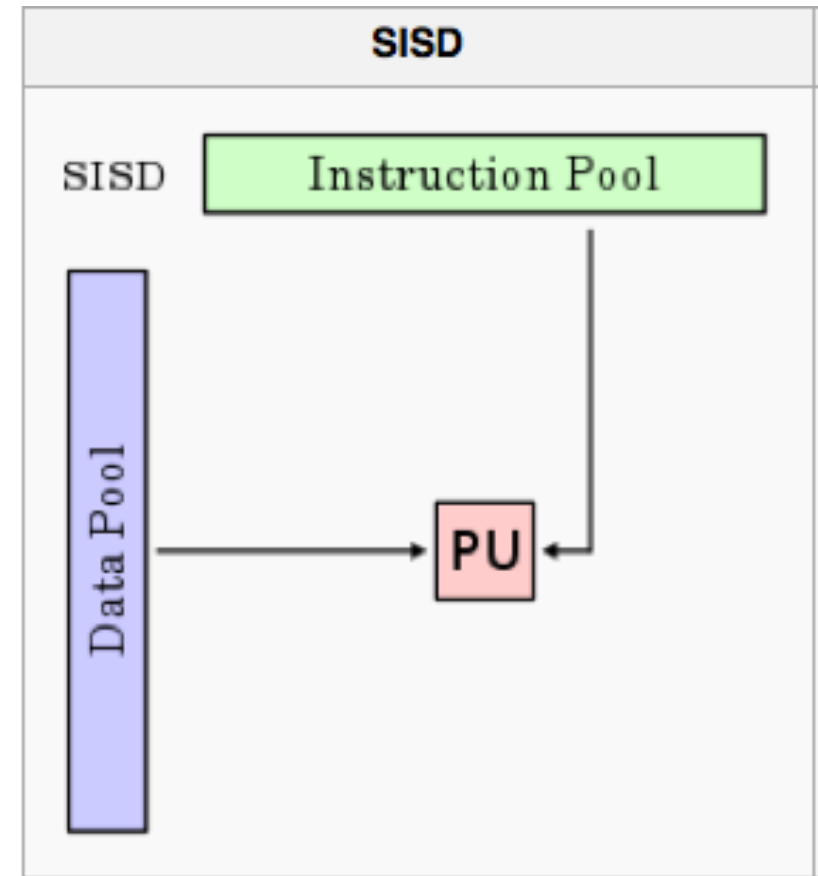
- Single Instruction, Single Data stream (SISD)
- Single Instruction, Multiple Data streams (SIMD)
- Multiple Instruction, Single Data stream (MISD)
- Multiple Instruction, Multiple Data streams (MIMD)

Flynn's taxonomy

	Single instruction	Multiple instruction
Single data	SISD	MISD
Multiple data	SIMD	MIMD

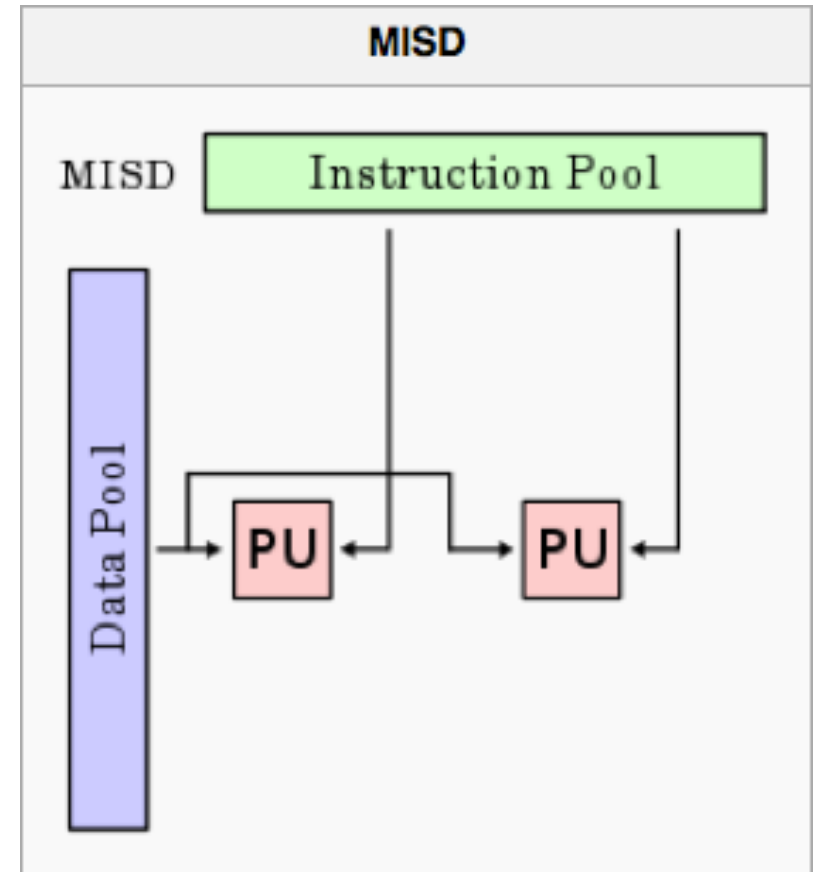
Flynn's::SISD Architectures

- Single Instruction, Single Data stream (SISD)
 - Sequential computer which exploits no parallelism in either the instruction or data streams
 - Single control unit (CU/CPU) fetches single Instruction Stream from memory. The CU/CPU then generates appropriate control signals to direct single processing element to operate on single Data Stream, i.e., one operation at a time.
 - Pretty much obsolete...!
 - Basic idea of von Neumann computer



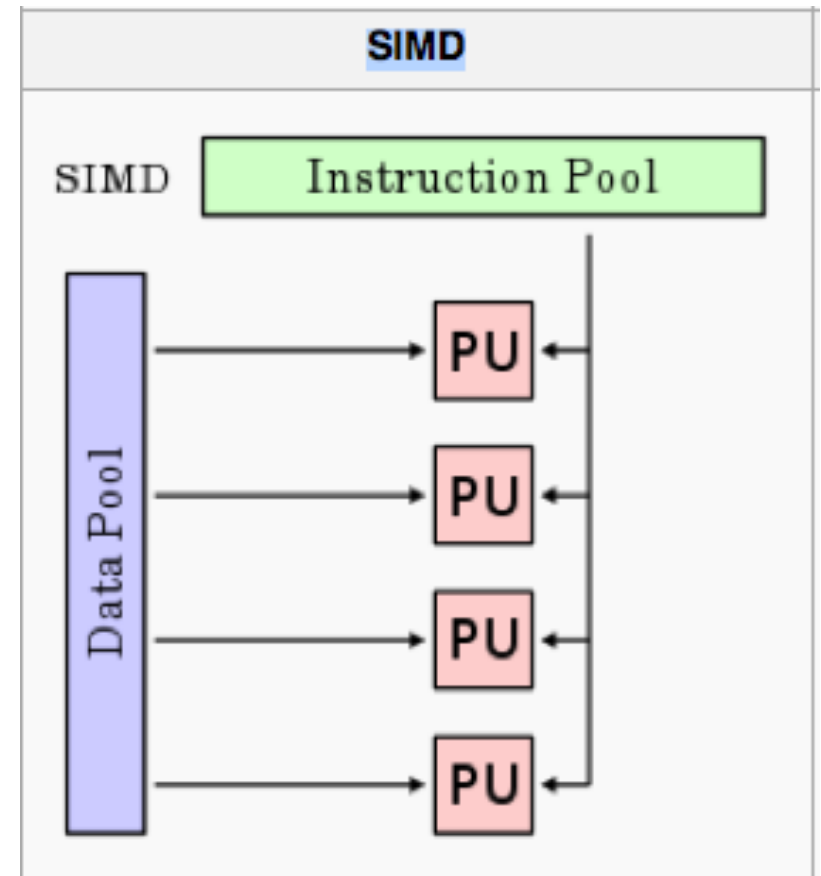
Flynn's::MISD Architectures

- Multiple Instruction, Single Data stream (MISD)
 - Parallel computing architecture where many functional units (PU/CPU) perform different operations on the same data
 - Examples include fault tolerant computer architectures, e.g., running multiple error checking processes on same data stream
 - (Not very common though!)



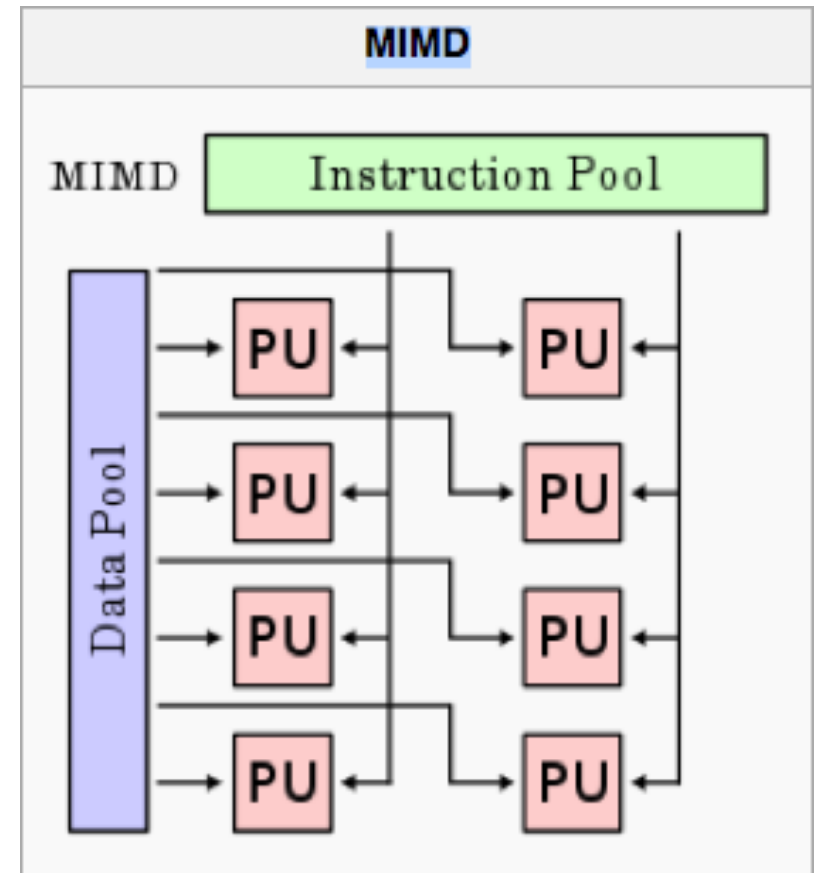
Flynn's::SIMD Architectures

- Single Instruction, Multiple Data stream (SIMD)
 - multiple processing elements that perform the same operation on multiple data points simultaneously
 - focus is on data level parallelism, i.e., many parallel computations, but only a single process (instruction) at a given moment
 - Many modern computers use SIMD instructions, e.g., to improve performance of multimedia use such as for image processing



Flynn's::MIMD Architectures

- Multiple Instruction, Multiple Data stream (MIMD)
 - number of processors that function asynchronously and independently
 - at any time, different processors may be executing different instructions on different pieces of data
 - machines can be shared memory or distributed memory categories
 - depends on how MIMD processors access memory
 - Most systems these days operate on MIMD
 - HPC ...



Approaches for Parallelism

- Where and how
 - Explicit vs Implicit parallelism
 - openMP / MPI
 - Hardware
 - Operating System
 - Software/Applications
 - Some or all of these

Explicit vs Implicit Parallelisation

- Implicit Parallelism
 - Supported by parallel languages and parallelizing compilers that take care of identifying parallelism, the scheduling of calculations and the placement of data
 - Pretty hard to do (more later!)
- Explicit Parallelism
 - In this approach, the programmer is responsible for most of the parallelization effort such as task decomposition, mapping tasks to processors, inter-process communications
 - This approach assumes user is the best judge of how parallelism can be exploited for a particular application
 - Typically, non-trivial to achieve!
 - Consider SPARTAN HPC cluster assignment
 - Slurm script (#SBATCH --ntasks=2 --cpus-per-task=4 vs
#SBATCH --ntasks=1 --cpus-per-task=8)*
 - (There is no MAGIC! More next week!)

Open Multi-Processing (OpenMP)

- Widely used API for shared-memory parallel programming
- Provides features that make it easier to write parallel programs for multicore and multiprocessor systems
 - Parallel Constructs
 - Define parallel regions where multiple threads execute code concurrently

```
#pragma omp parallel  
{ // Code executed by multiple threads }
```

- Work Sharing Constructs
 - Mechanisms to divide work among threads:

#pragma omp for : Distributes loop iterations among threads

#pragma omp sections: Assigns different code sections to different threads

#pragma omp single: Ensures a block of code runs on a single thread

#pragma omp task: Dynamically creates tasks for threads to execute

OpenMP...ctd

- **Thread Management**

- *omp_set_num_threads()* sets the number of threads
- *omp_get_num_threads()* for thread counts
- *omp_get_thread_num()* for thread IDs

- **Synchronization Constructs**

- Critical Sections (**#pragma omp critical**): only one thread executes block of code at time
- Atomic Operations (**#pragma omp atomic**): lightweight synchronization of simple opns.
- Barrier (**#pragma omp barrier**): Synchronizes all threads at specific point in program
- Locks: *omp_set_lock()* and *omp_unset_lock()* provide finer control over mutual exclusion

- **Data Sharing Attributes**

- Shared: All threads access the same instance of a variable
- Private: Each thread has its own instance of a variable
- Firstprivate: Each thread gets a private copy of a variable, initialized by master thread
- Reduction: Combines thread-local copies of a variable into a single value

- **Loop Scheduling**

- Static scheduling: Predefined assignment of iterations to threads.
- Dynamic scheduling: Threads take chunks of iterations dynamically
- Guided scheduling: Threads take decreasing chunks of iterations
- Auto: Delegates scheduling to the compiler/runtime

- **Portable and Scalable**

- Designed to be portable across different HW/OS, C, C++, Fortran, Python, ...

Message Passing Interface (MPI)

- Widely adopted approach for message passing in parallel systems
- Mappings to major languages Fortran, C, C++, Python, Java
 - Standardised, widely adopted, portable, performant, ...
 - Parallelisation = user's problem
- Key MPI functions
 - MPI_Init :initiate MPI computation
 - MPI_Finalize :terminate computation
 - MPI_COMM_SIZE :determine number of processors
 - MPI_COMM_RANK :determine my process identifier
 - MPI_SEND :send a message
 - MPI_RECV :receive a message
- Supports point-point, broadcast communications
 - Examples of programming MPI next week (for assignment)

HelloWorld MPI

```
/* C Example */
#include <stdio.h>
#include <mpi.h>
int main (argc, argv)
int argc;
char *argv[];
{
    int rank, size;
    MPI_Init (&argc, &argv); /* starts MPI */
    MPI_Comm_rank (MPI_COMM_WORLD, &rank); /* get current process id */
    MPI_Comm_size (MPI_COMM_WORLD, &size); /* get number of processes
*/
    printf( "Hello world from process %d of %d\n", rank, size );
    MPI_Finalize();
    return 0;
}
```

```
$mpicc -o hello_world hello_world.c
```

```
$mpirun -np 4 hello_world %np passed in as part of SLURM script
```

```
Hello world from process 1 of 4
Hello world from process 2 of 4
Hello world from process 3 of 4
Hello world from process 0 of 4
```

Approaches for Parallelism

- Where and how
 - Explicit vs Implicit parallelism
 - openMP / MPI
 - Hardware
 - Operating System
 - Software/Applications
 - Some or all of these

(Heavily Simplified) Hardware Parallelisation

Basic
CPU

Hardware
Threading CPU

Cache

Cache

Control Unit

Control Unit

Control Unit

Integer

Floating
Point

Vector

Integer

Floating
Point

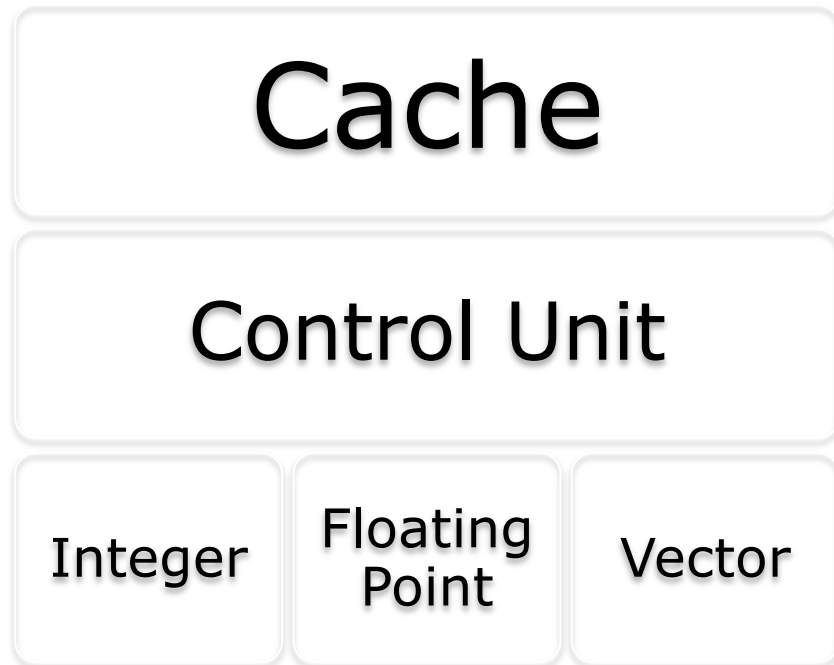
Vector

Cache – much faster than reading/writing to main memory; instruction cache, data cache (multi-level) and translation lookaside buffer used for virtual-physical address translation (more later on Cloud and hypervisors).

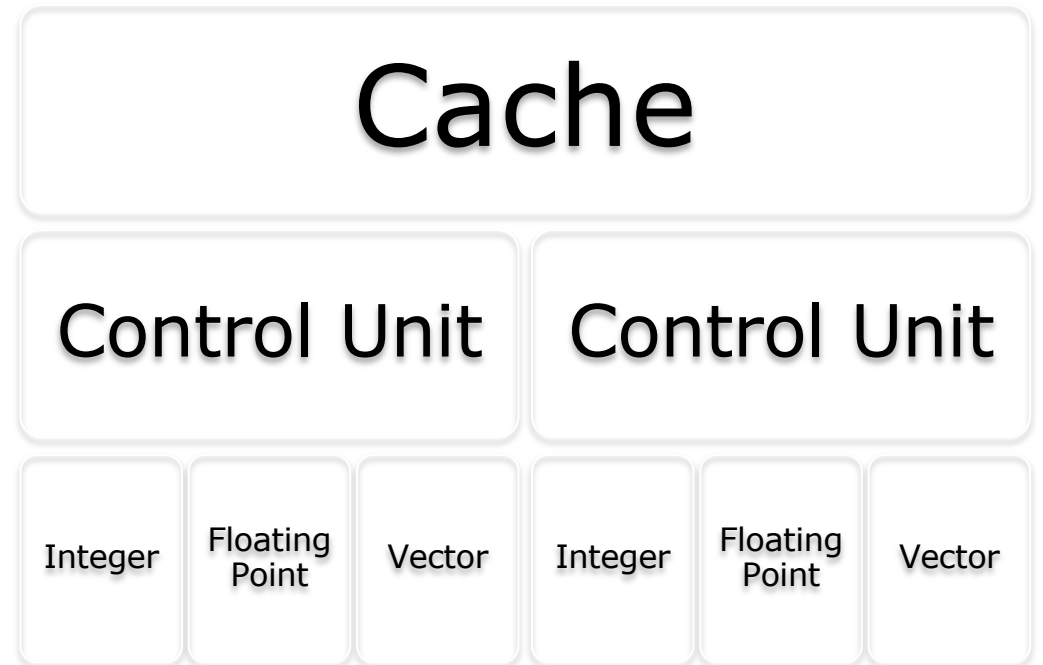
Parallelisation by adding extra CPU to allow more instructions to be processed per cycle. Usually shares arithmetic units. Heavy use of one type of computation can tie up all the available units of the CPU preventing other threads from using them.

(Heavily Simplified) Hardware Parallelisation

Basic CPU

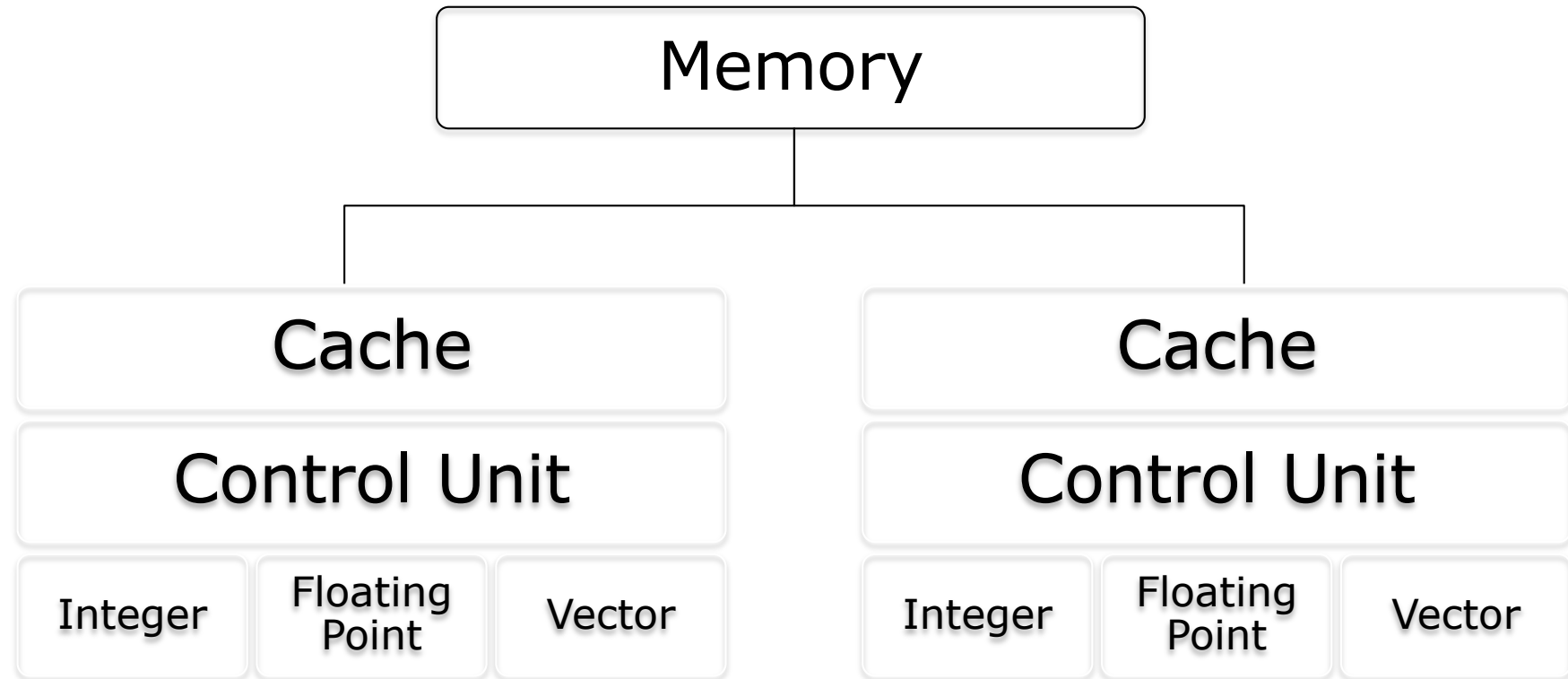


Multi-Core



Multiple cores that can process data and (*in principle!!!*) perform computational tasks in parallel. Typically share same cache, but issue of cache read/write performance and cache coherence. Possibility of cache stalls (CPU not doing anything whilst waiting for caching); many chips have mixture (L1 cache on single cores; L2 cache on pairs of cores; L3 cache shared by all cores); typical to have different cache speeds and cache sizes (higher hit rates but potentially higher latency).

Heavily Simplified (Symmetric Multiprocessing (SMP))



Two (or more) identical processors connected to a single, shared main memory, with full access to all I/O devices, controlled by a single OS instance that treats all processors equally. Each processor executes different programs and works on different data but with capability of sharing common resources (memory, I/O device, ...). Processors can be connected in a variety of ways: buses, crossbar switches, meshes. More complex to program since need to program both for CPU and inter-processor communication (bus).

Approaches for Parallelism

- Where and how
 - Explicit vs Implicit parallelism
 - openMP / MPI
 - Hardware
 - Operating System
 - Software/Applications
 - Some or all of these

Operating System Parallelism Approaches

- Most modern multi-core operating systems support different “forms” of parallelisation
 - parallel vs interleaved semantics
 - $A \parallel B$ vs $A \parallel \parallel B$
- Compute parallelism
 - Processes
 - Used to realise tasks, structure activities, ...
 - Threads
 - Native threads
 - Fork, Spawn, Join
 - Green threads
 - Scheduled by a virtual machine instead of natively by the OS
- Data parallelism
 - Caching (cache coherency)
 - OS implies on “a” computer

Approaches for Parallelism

- Where and how
 - Explicit vs Implicit parallelism
 - openMP / MPI
 - Hardware
 - Operating System
 - Software/Applications
 - Some or all of these

Software Parallelism Approaches

- Many (most!) languages now support a range of parallelisation/concurrency features
 - Threads, thread pools, locks, semaphores, ...
- Many languages developed specifically for parallel/concurrent systems
 - http://en.wikipedia.org/wiki/Concurrent_computing
- Key issues that need to be tackled
 - Deadlock
 - processes involved constantly waiting for each other
 - Livelock
 - processes involved in livelock constantly change with regard to one another, but none are progressing

Process P	Process Q
...	...
Get A	Get B
...	...
Get B	Get A
...	...
Release A	Release B
...	...
Release B	Release A
...	...

(HT)Condor



- A specialized workload management system for compute-intensive jobs developed at University of Wisconsin
 - Offers job queueing mechanisms, scheduling policies, priority schemes, resource monitoring/management
 - User submits jobs to Condor, and it chooses when and where to run the jobs, monitors their progress, and informs the user upon completion
 - Allows to harvest “free” (?) CPU power from otherwise idle desktop workstations
 - e.g. use desktop machines when keyboard and mouse are idle
 - key press detected checkpoint and migrate a job to a different (idle) machine
 - No need for shared file system across machines
 - Data can be staged to machines as/when needed
 - Can work across organisational boundaries
 - Condor Flocking
 - ClassAds
 - Advertise resources and accept jobs (according to policy)

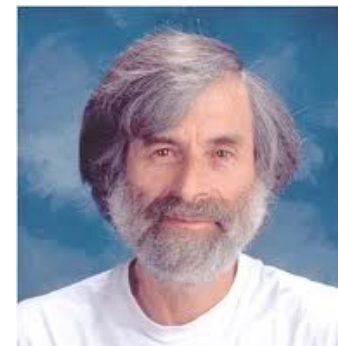
Data Parallelism Approaches

- Challenges of big data
 - The most important kind of parallelism challenge?
- Distributed data
 - Consistency, Availability, Partition tolerance
 - CAP Theorem – more later
 - ElasticSearch (clustered database solutions)
 - ACID <-> BASE
- Distributed File Systems
 - e.g. Hadoop, Lustre, Ceph...

Break

Challenges with Distribution

- *A distributed system is one in which the failure of a computer you didn't even know existed can render your own computer unusable*
- *Leslie Lamport*



Challenges with Distribution

- *General assumptions that typically don't hold in the real world...*

(Some) Erroneous Assumptions of Distributed Systems

1. The network is reliable
2. Latency is zero
3. Bandwidth is infinite
4. The network is secure
5. Topology doesn't change
6. There is one administrator
7. Transport cost is zero
8. The network is homogeneous
9. Time is ubiquitous

Network reliability

- If I send some data over the network
 - It will arrive
 - It will arrive in the order I sent it
 - It will arrive uncorrupted
- If the network is unreliable, it will be so in only one of the above ways
 - Consistently
- The lower layers in the networking stack protect me from these issues
- None of these statements are always true!

Erroneous Assumptions of Distributed Systems

1. The network is reliable
2. Latency is zero
3. Bandwidth is infinite
4. The network is secure
5. Topology doesn't change
6. There is one administrator
7. Transport cost is zero
8. The network is homogeneous
9. Time is ubiquitous

Latency is zero

- If I send some data, it will arrive “now”.
 - Or so fast to be effectively “now”
- Really...?
- Consider accessing Twitter (San Francisco) from Melbourne
 - ~12,600km

The Route (traceroute)

1.	128.250.7.66	0.646ms
2.	172.18.65.89	1.135ms
3.	fr151-v2010-vpn-general.unimelb.net.au	1.203ms
4.	fr151-v2011-vpn-fusion-general.unimelb.net.au	0.895ms
5.	br-151-te-2-3.unimelb.net.au	4.696ms
6.	gigabitethernet1.er1.unimelb.cpe.aarnet.net.au	0.887ms
7.	ge-5-1-0.bb1.a.mel.aarnet.net.au	1.223ms
8.	so-0-1-0.bb1.a.syd.aarnet.net.au	15.158ms
9.	so-2-2-0.bb1.a.pao.aarnet.net.au	171.461ms
10.	paix1.cr1.pao1.twttr.com	172.682ms
11.	ae52.smf1-er1.twttr.com	177.191ms

Latency Consequences

- On a 1Gbps link
 - ~9MB of data can be in flight depending on TCP packet size
 - You could have sent 18MB of data before you start receiving responses, all of which will be in your TCP stack's buffer
- Latency can be much higher
- In a system with many nodes/hops each link can have different latencies
 - Next hop behaviour can be challenging
 - Explored in active networking technologies, software defined networking, ...

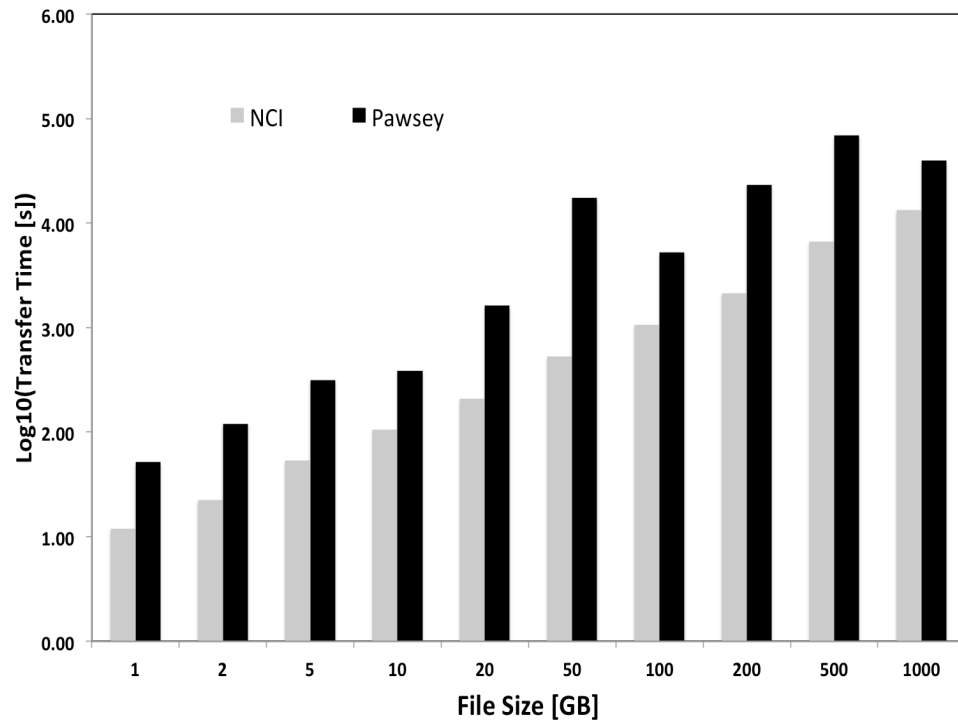
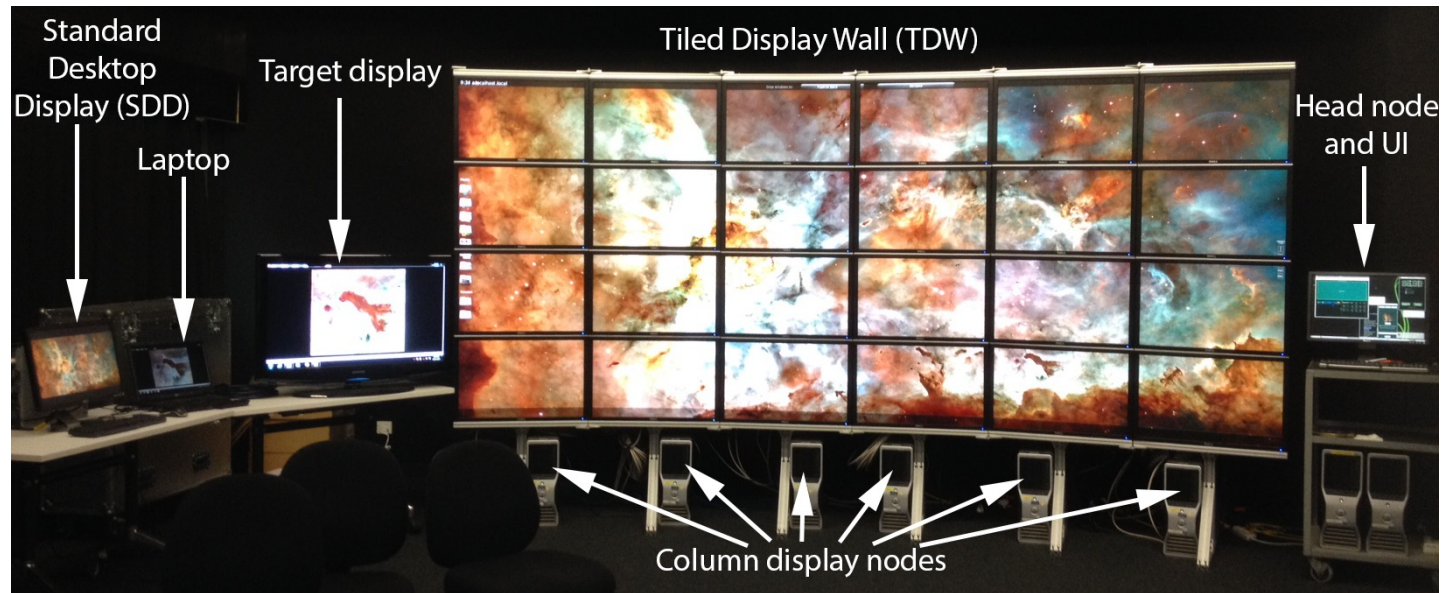
Erroneous Assumptions of Distributed Systems

1. The network is reliable
2. Latency is zero
3. Bandwidth is infinite
4. The network is secure
5. Topology doesn't change
6. There is one administrator
7. Transport cost is zero
8. The network is homogeneous
9. Time is ubiquitous

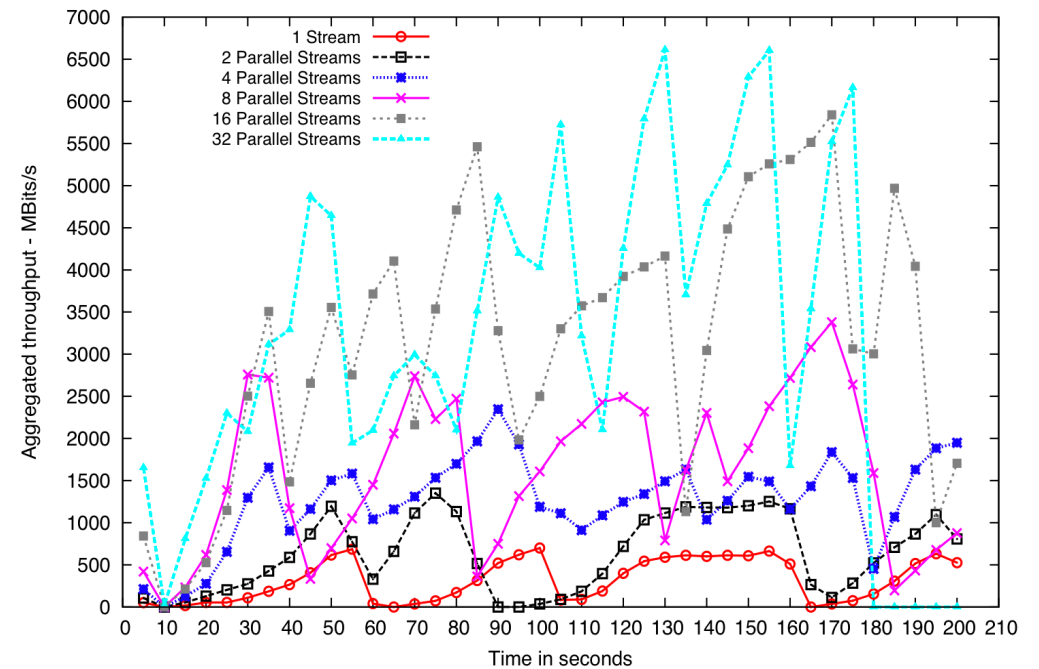
Bandwidth is infinite

- I can send any amount of data I wish between any nodes
- Really?
 - Think of BIG DATA (Tb->Pb+)
 - SKA, LHC, Genomics, ...
 - And consequences for others, e.g., Melbourne hospitals
 - Network bandwidth typically of the order
 - 1GB, 10GB, 100GB, ...
 - For hospitals, homes varies wildly...
 - Networking capacity carefully planned
 - JANET (UK) backbone planned for up to 25% actual bandwidth capacity actually used

Bandwidth is infinite



IPerf 10Gb Test (TCP window == 32MBytes) - Pawsey (Perth) to MIT (Boston) - throughput series sampled every 5 seconds



Erroneous Assumptions of Distributed Systems

1. The network is reliable
2. Latency is zero
3. Bandwidth is infinite
4. The network is secure
5. Topology doesn't change
6. There is one administrator
7. Transport cost is zero
8. The network is homogeneous
9. Time is ubiquitous

The network is secure

- Really...?
- So, I don't have to worry about...
 - People sending data to my services
 - Repeated password attempts, SQL injections, ...!?
 - People actively attacking me
 - Distributed denial of service attacks
 - People reading the data sent over the network
 - Man in the middle attacks
 - People masquerading as one of my nodes
 - Spoofing
 - People breaking into one of my nodes
 - Trojans, viruses, brute force attacks, ...
 - People stealing the physical hardware

Erroneous Assumptions of Distributed Systems

1. The network is reliable
2. Latency is zero
3. Bandwidth is infinite
4. The network is secure
5. Topology doesn't change
6. There is one administrator
7. Transport cost is zero
8. The network is homogeneous
9. Time is ubiquitous

Topology doesn't change

- Node x is always *there*
- Really?
 - There = ?
 - Latency
 - IP
 - Route
 - Services
 - Typically, can't guarantee the route taken and hence latency
 - unless specific routing protocols selected, e.g., differentiated services with reservation protocols
 - Mostly not available (TCP/IP)
 - Next hop behaviour again!

Erroneous Assumptions of Distributed Systems

1. The network is reliable
2. Latency is zero
3. Bandwidth is infinite
4. The network is secure
5. Topology doesn't change
6. There is one administrator
7. Transport cost is zero
8. The network is homogeneous
9. Time is ubiquitous

There is one administrator

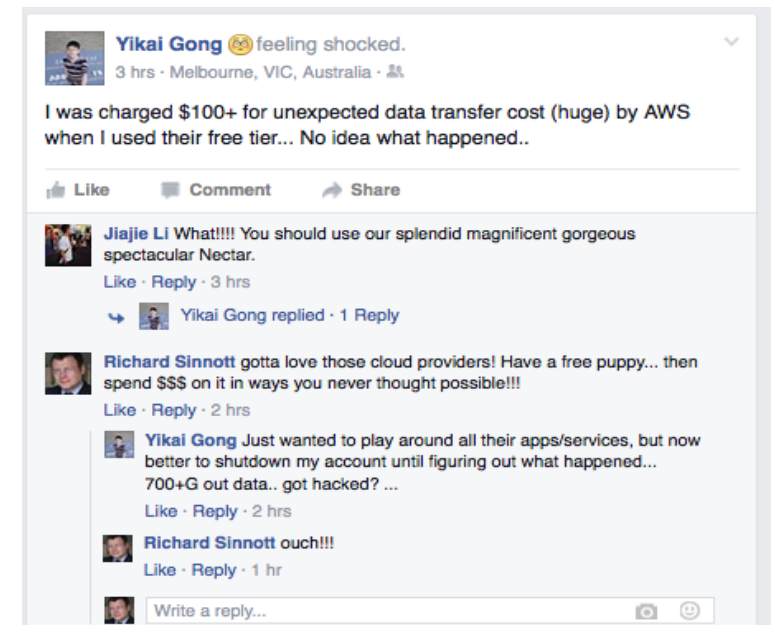
- Really?
 - Pretty much never the case
 - Who is administrator for University of Melbourne
 - Firewall changes, server reconfigurations, services, access control (students/staff/others...)
 - Inter-organisational administrators?

Erroneous Assumptions of Distributed Systems

1. The network is reliable
2. Latency is zero
3. Bandwidth is infinite
4. The network is secure
5. Topology doesn't change
6. There is one administrator
7. Transport cost is zero
8. The network is homogeneous
9. Time is ubiquitous

Transport cost is zero

- I can send as much data as I like for free
- Really?
 - Can appear so in some places, but...
 - Australia...!?!?!?
 - Capped uploads/downloads
 - UniMelb researchers with \$50k bill for data access that wasn't over AARNET
 - Amazon EC2/S3...\$\$\$\$\$
 - It is NEVER free!



Erroneous Assumptions of Distributed Systems

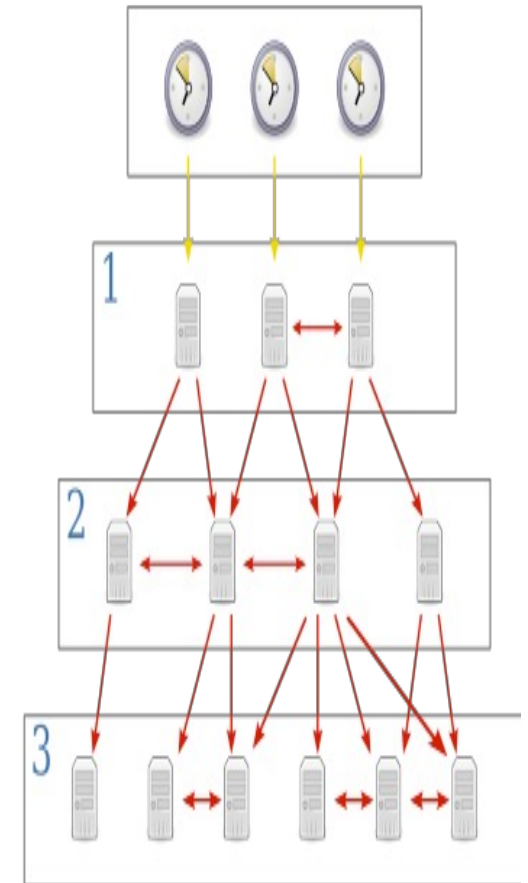
1. The network is reliable
2. Latency is zero
3. Bandwidth is infinite
4. The network is secure
5. Topology doesn't change
6. There is one administrator
7. Transport cost is zero
8. The network is homogeneous
9. Time is ubiquitous

Erroneous Assumptions of Distributed Systems

1. The network is reliable
2. Latency is zero
3. Bandwidth is infinite
4. The network is secure
5. Topology doesn't change
6. There is one administrator
7. Transport cost is zero
8. The network is homogeneous
9. Time is ubiquitous

Time is ubiquitous

- The clock time is the same across all computers in the network
- Really?
 - Clocks, clock cycles
 - Clock drift, skew, ...
 - Protocols
 - NTP synchronizes participating computers to **within a few milliseconds** of Coordinated Universal Time (UTC)
 - Spaghetti westerns
 - Lip-synchronisation algorithms
 - Many approaches to reduce latency, jitter, ...
 - For some applications this is a MAJOR issue, e.g., finance!



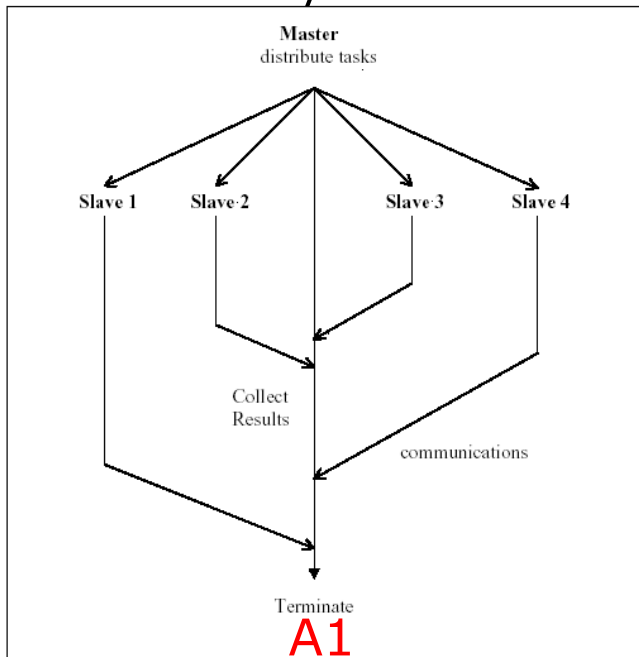
Despite this...

- ...and issues of heterogeneity of compute, data, security, ...
 - (Remember lecture 1)
- Distributed systems are widespread
 - The Internet!
- Designing parallel or distributed systems
 - Many approaches
 - No single algorithm
 - No single technical solution
 - Eco-system of approaches explored over time and many open research questions/challenges

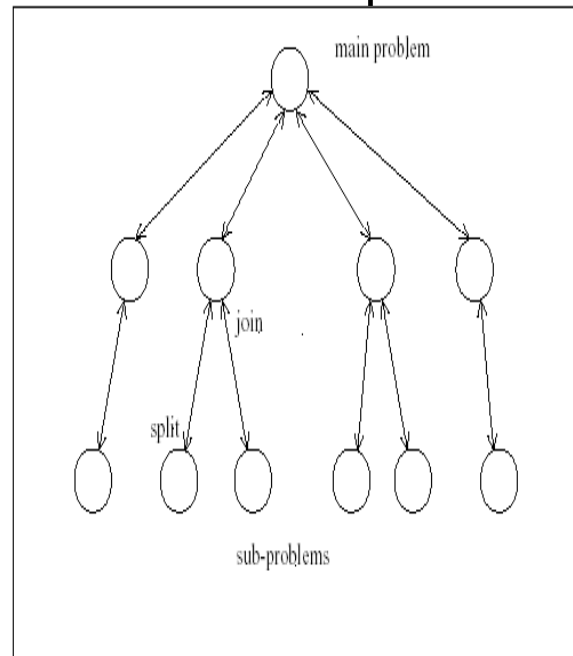
Recipes/Patterns for Distributed Processing

- Many approaches

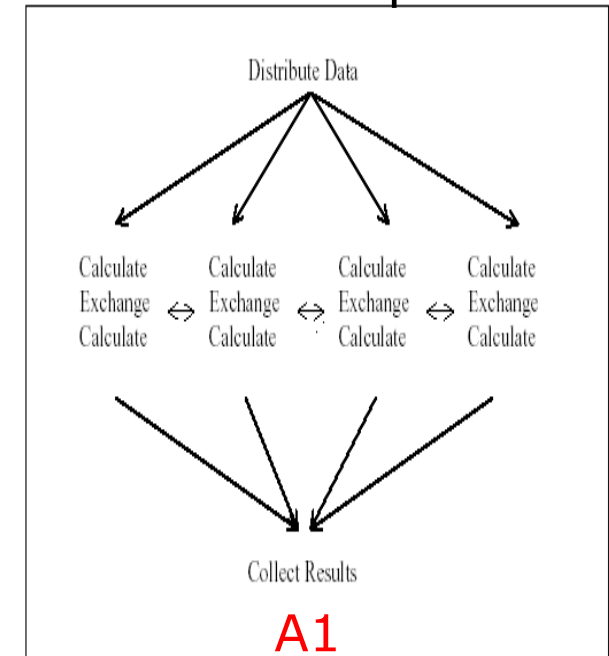
Master/slave



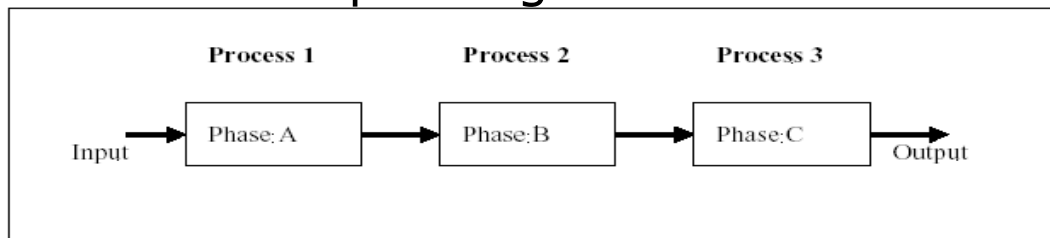
Task Decomposition



Data Decomposition



Data Pipelining



Summary

- Lots of open issues with design, implementation and verification of distributed and parallel systems
- Hardware refinements/evolution still happening
 - FPGAs, GPUs, ...
 - Flavours of data storage
- Software challenges/algorithms
- Design issues – often challenged by domain needs
 - Reliability vs performance
 - Security vs scale
 - ...

Assignment 1

- Use Karaage to create your account asap
- ALWAYS “sbatch” (more next week)
 - Else you are running on the login node and bad things will happen to it (and you!!!)
- Suggest to use Python or C or C++
 - If you can only program in Java then look at MPJ
 - But 1 vs 2 node scenario
- Big tip
 - Work out how to process data without reading it all into memory
 - You can get away with this for the smaller files, but not for the big files!
 - (Welcome to the big data world!!!)
 - *Big data* = if you can process it on a single server then it ain't that big!

Questions